

Chapter 2: Programming Architecture

2.1 Model View Controller (MVC)

In software engineering, a design pattern is a general reusable solution to a commonly occurring problem within a given context in software design. A design pattern is not a finished design that can be transformed directly into source or machine code. It is a description or template for how to solve a problem that can be used in many different situations. Object-oriented design patterns typically show relationships and Interactions between classes or objects, without specifying the final application classes or objects that are involved. One of these design patterns is Model-View-Controller (MVC). The programming language Smalltalk first defined the MVC concept it in the 1970's. Since that time, the MVC design idiom has become commonplace, especially in object-oriented systems.

The Architecture

In a MVC application as having three main layers: **presentation (UI), application logic, and resource management**. In MVC, the presentation layer is split into controller and view. The most important separation is between presentation and application logic. The View/Controller split is less so. MVC encompasses more of the architecture of an application than is typical for a design pattern. Hence the term architectural pattern may be useful, or perhaps an aggregate design pattern.

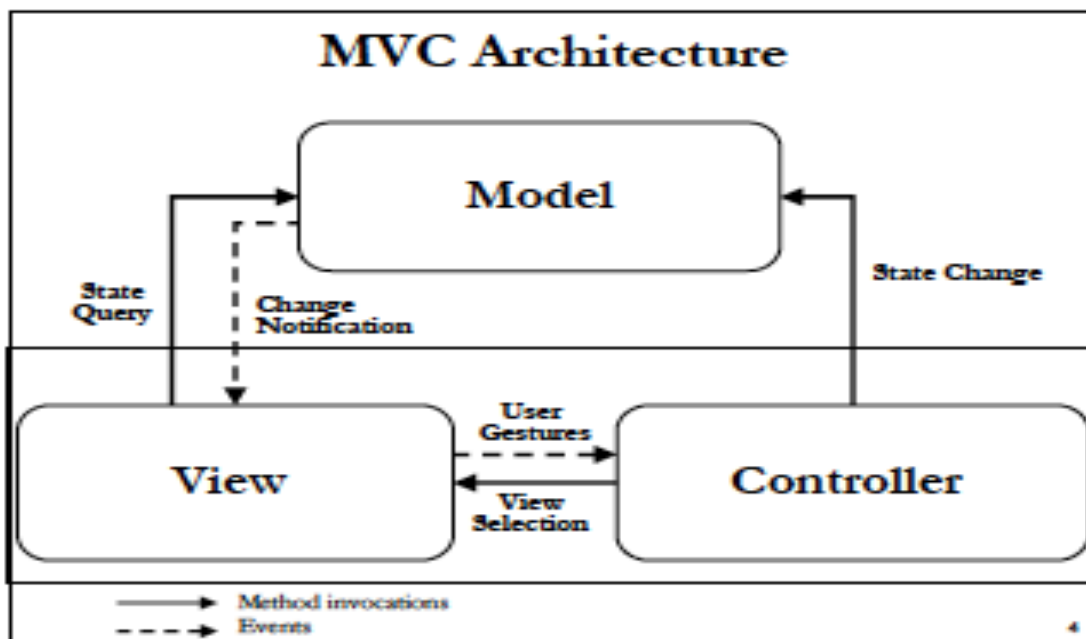


Fig: Model View Controller

1. **Model:** The domain-specific representation of the information on which the application operates. The model is another name for the application logic layer (sometimes also called the domain layer). Application (or domain) logic adds meaning to raw data (e.g., calculating if today is the user's birthday, or the totals, taxes and shipping charges for shopping cart items). Many applications use a persistent storage mechanism (such as a database) to store data. MVC does not specifically mention the resource management layer because it is understood to be underneath or encapsulated by the Model.
2. **View:** Renders the model into a form suitable for interaction, typically a user interface

element. MVC is often seen in web applications, where the view is the HTML page and the code which gathers dynamic data for the page.

3. **Controller:** Processes and responds to events, typically user actions, and may invoke changes on the model and view.

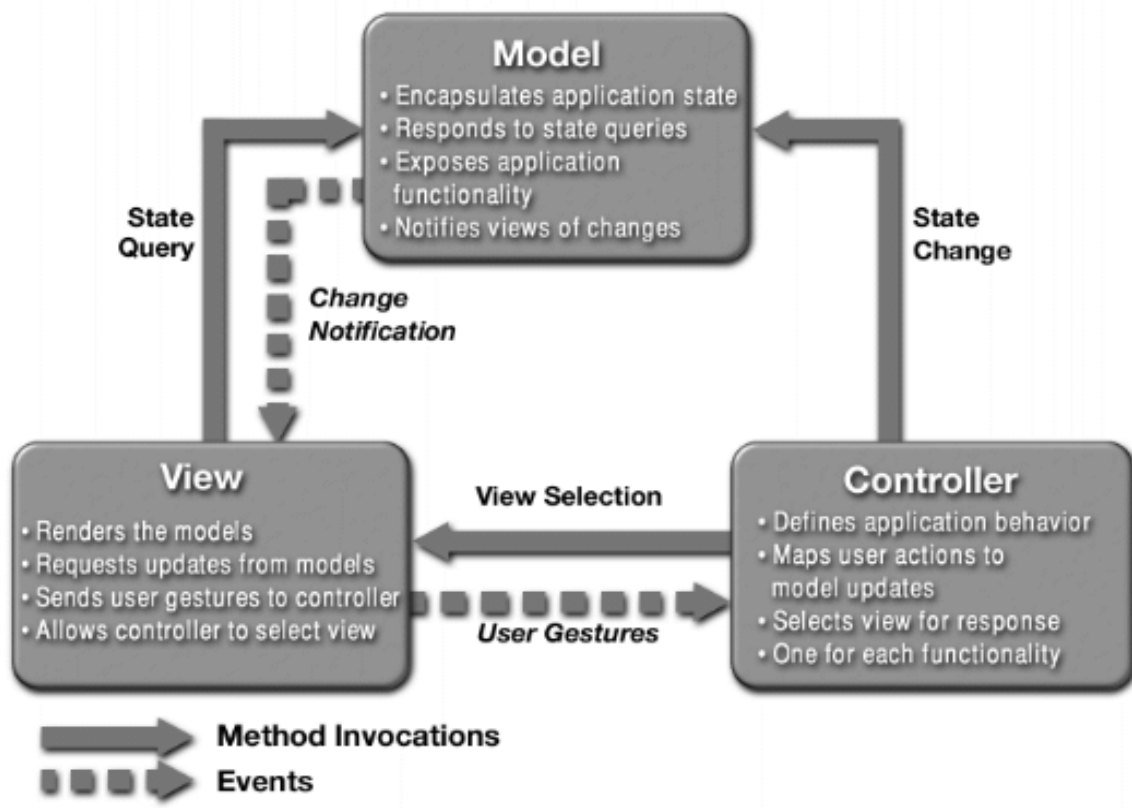


Fig: Summary of relationship between Model, View and Controller

Though MVC comes in different flavours, the control flow generally works as follows:

1. The user interacts with the user interface in some way (e.g., user presses a button)
2. A controller handles the input event from the user interface, often via a registered handler or callback.
3. The controller accesses the model, possibly updating it in a way appropriate to the user's action (e.g., controller updates user's shopping cart). Complex controllers are often structured using the command pattern to encapsulate actions and simplify extension.
4. A view uses the model to generate an appropriate user interface (e.g., view produces a screen listing the shopping cart contents). The view gets its own data from the model. The model has no direct knowledge of the view. (However, the observer pattern can be used to allow the model to indirectly notify interested parties, potentially including views, of a change.)
5. The user interface waits for further user interactions, which begins the cycle anew.

2.2 Client Server Model

Early computer system can best be described as monolithic system in which all processing occurred on the same machine. User interacted with the monolithic system through dumb terminal. It is desirable to move away from such monolithic system and divide the work between several different computers- this change signaled the birth of client/server systems.

All major software system relies on a persistent storage mechanism for maintaining state between program invocations. In such system, it is possible to clearly separate processing into two distinct areas

1. Processing that supports the storage and retrieval of persistent data as well as concurrent access to persistent storage.
2. All other processing required within software system

Splitting the work among multiple machines was not the motivation for dividing client/server technology. **Business data** is the most important asset that most companies possess. Maintaining database on server separate from the client allows for **greater security, reliability and performance** with regard to data storage while simultaneously reducing the high cost associated with monolithic system.

The client-server model is a distributed application structure in computing that partitions tasks or workloads between the providers of a resource or service, called servers, and service requesters, called clients. Often clients and servers communicate over a **computer network** on separate hardware, but both client and server may reside in the same system. A server is a host that is running one or more server programs which share their resources with clients. A **client does not share any of its resources**, but **requests a server's content** or service function. Clients therefore initiate communication sessions with servers that await incoming requests.



Fig: Client Server Model

Types of Server

There are two types of servers you can have:

1. **Iterative Server:** This is the simplest form of server where a server process serves one client and after completing first request then it takes request from another client. Meanwhile another client keeps waiting.
2. **Concurrent Servers:** This type of server runs multiple concurrent processes to serve many request at a time. Because one process may take longer and another client cannot wait for so long.

2.2.1. 2-tier Client Server Architecture

In this architecture, client directly interacts with the server. 2-tier architecture is used to describe client/server systems where the client requests resources and the server responds directly to the request, using its own resources. This means that the server does not call on another application in order to provide part of the service.

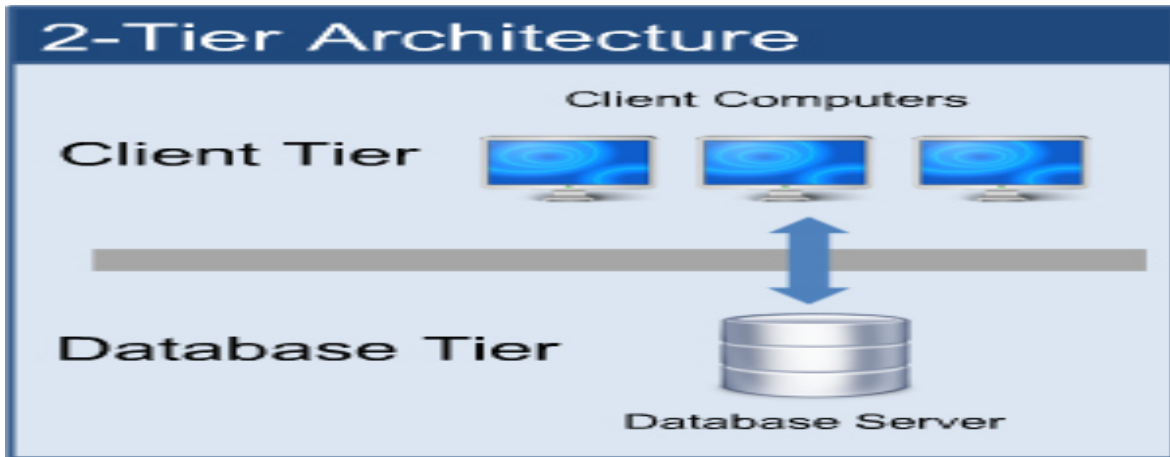
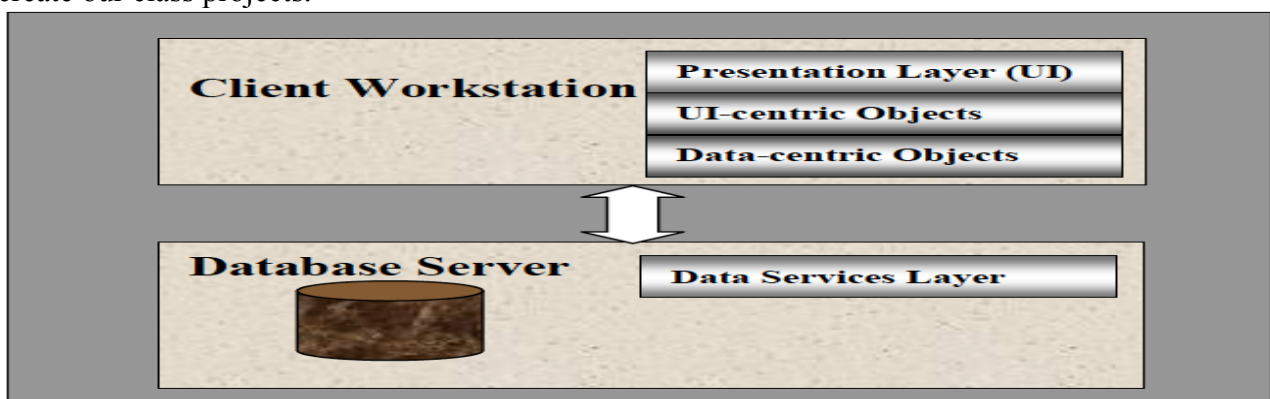


Fig: 2-tier architecture

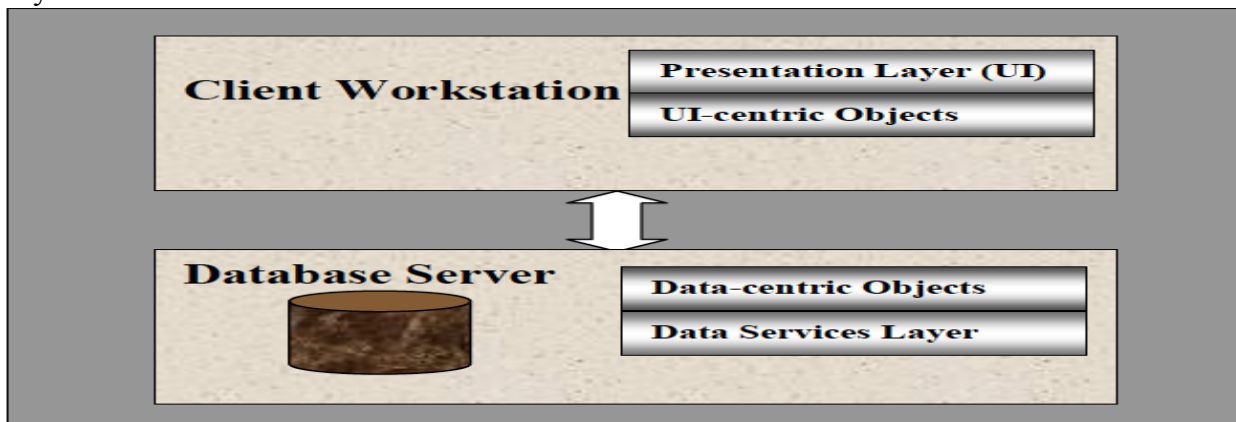
2-tier Pros and Cons

Advantages	Disadvantage
<i>Development Issues:</i> • Simple structure • Easy to setup and maintain	<i>Development Issues:</i> • Complex application rules difficult to Implement in database server – requires more code for the client • Complex application rules difficult to implement in client and have poor performance • Changes to business logic not automatically enforced by a server – changes require new client side software to be distributed and installed
<i>Performance:</i> • Adequate performance for low to medium volume environments • Business logic and database are physically close, which provides higher performance.	<i>Performance:</i> • Inadequate performance for medium to high volume environments, since database server is required to perform business logic. This slows down database operations on database server.

Fat-Client/Fat-Server: Client does most of the processing. User Interface and all data processing/Business logic are done on client. The Data Service Layer is the actual Database Server, which handles the storage or services the data. Note that this will be the architecture we will use to create our class projects.



Thin-Client/Fat-Server: Client has less processing, usually UI & UI-Centric *Business Objects*. The *Data-Centric Business Objects*, which handles the data access code, resides on the Data Service Layer or Database Server.



2.2.2. 3-tier Architecture

In this architecture, one more software sits in between client and server. This middle software is called **middleware**. Middleware are used to perform all the security checks and load balancing in case of heavy load. A middleware takes all requests from the client and after doing required authentication it passes that request to the server. Then server does required processing and sends response back to the middleware and finally middleware passes this response back to the client.

Middle tier is also called as business logic tier or service tier. In the simple client-server solution the client was handling the business logic and that makes the client “**thick**”. A thick client means that it requires heavy traffic with the server, thus making it difficult to use over slower network connections like Internet and Wireless (LTE, 3G, or Wi-Fi).

By introducing the middle layer, the client is only handling presentation logic. This means that only little communication is needed between the client and the middle tier making the client “**thin**” or “**thinner**”. An example of a thin client is an Internet browser that allows you to see and provide information fast and almost with no delay.

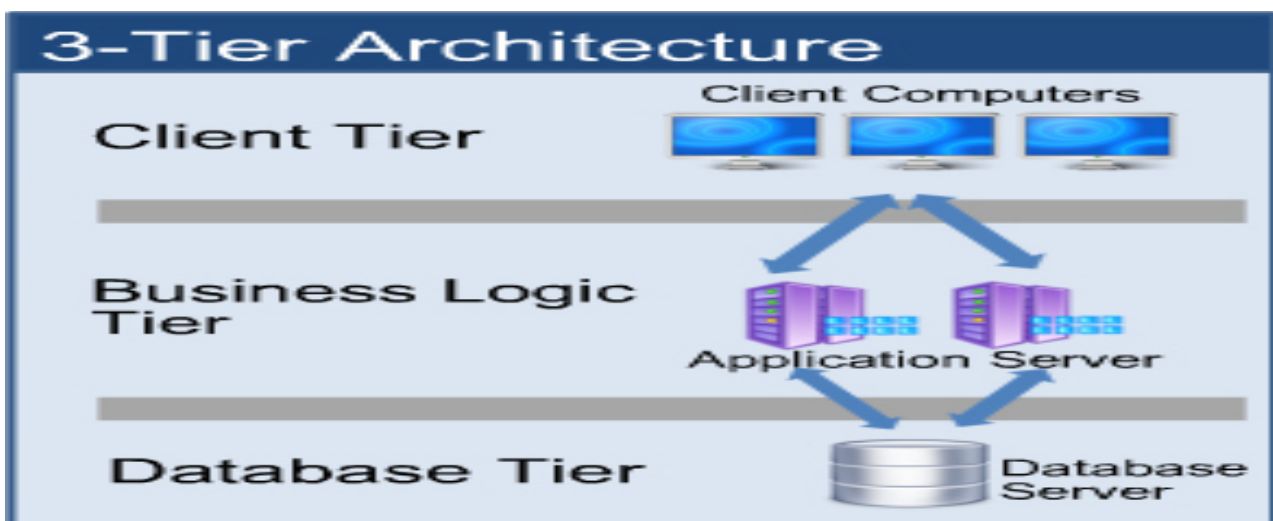


Fig: 3-tier architecture

3-tier pros and cons

Advantage	Disadvantage
<i>Development Issues:</i> • Complex application rules easy to implement in application server • Business logic off-loaded from database server and client, which improves performance • Changes to business logic automatically enforced by server – changes require only new application server software to be installed • Application server logic is portable to other database server platforms by virtue of the application software	<i>Development Issues:</i> • More complex structure • More difficult to setup and maintain.
<i>Performance:</i> • Superior performance for medium to high volume environments.	<i>Performance:</i> • The physical separation of application servers containing business logic functions and database servers containing databases may moderately affect performance.

2.2.3. N-tier Architecture

As more users access the system a three-tier solution is more scalable than the other solutions because we can add as many middle tiers (running on each own server) as needed to ensure good performance (N-tier or multiple-tier). Security is also the best in the three-tier architecture because the middle layer protects the database tier. There is one major drawback to the N-tier architecture and that is that the additional tiers increase the complexity and cost of the installation.

In software engineering, multi-tier architecture (often referred to as n-tier architecture) is client-server architecture in which, the **presentation**, the **application processing** and the **data management** are logically separate processes. For example, an application that uses middleware to service data requests between a user and a database employs multi-tier architecture. The most widespread use of "multi-tier architecture" refers to three-tier architecture.

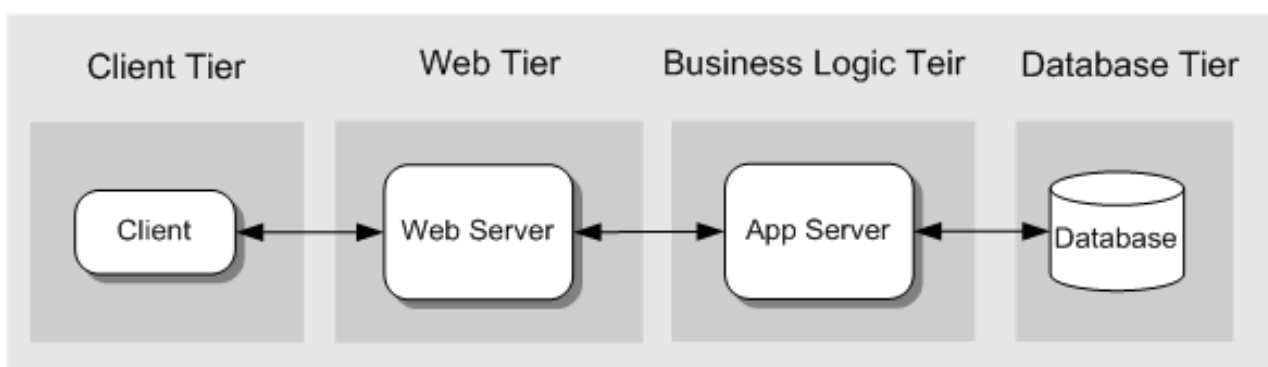


Fig: An example of N-tier Architecture

An example of N-tier architecture is web-based application. A web-based application might consist of the following tiers.

1. **Tier 1:** a client tier presentation implemented by a web browser.
2. **Tier 2:** a middle-tier distribution mechanism implemented by a web server.

3. **Tier 3:** a middle-tier service implemented by a set of server side scripts.
4. **Tier 4:** a data-tier storage mechanism implemented by a relational database.

2.3. Comparison between different Client Server Models

	1-tier	2-tier	3-tier (N-tier)
Benefits	Very simple Inexpensive No Server needed	Good Security More Scalable Faster execution	Exceptional security Faster execution Thin client Very scalable
Issues	Poor security Multi user issues	More costly More complex Thick client	Very costly Very complex
Users	Usually 1(or few)	2-100	50-2000(+)

2.4 Client Server versus MVC

1. **Communication:** A fundamental rule in a three-tier architecture is the **client tier never communicates directly with the data tier**; in a three-tier model all communication must pass through the middle tier. Conceptually the three-tier architecture is linear.

However, the MVC architecture is triangular: **the view sends updates to the controller, the controller updates the model, and the view gets updated directly from the model.**

2. **History:** From a historical perspective the three-tier architecture concept emerged in the 1990s from observations of distributed systems (e.g., web applications) where the client, middleware and data tiers ran on physically separate platforms.

Whereas MVC comes from the previous decade (by work at Xerox PARC in the late 1970s and early 1980s) and is based on observations of applications that ran on a single graphical workstation; MVC was applied to distributed applications later in its history.

Today, MVC and similar model-view-presenter (MVP) are Separation of Concerns design patterns that apply exclusively to the presentation layer of a larger system. In simple scenarios MVC may represent the primary design of a system, reaching directly into the database; however, in most scenarios the Controller and Model in MVC have a loose dependency on either a Service or Data layer/tier. This is all about Client-Server architecture in the 3-tier equivalent, communication between layers is bi-directional and always passes through the Middle tier in the MVC equivalent the communication is in unidirectional; we could say that each "layer" is updated by the one at the left and, in turn, updates the one at the right –where "left" and "right" are merely illustrative